

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NAMA : Raka Yulia Vegora

NIM : 109082500201

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Searching atau pencarian adalah proses menemukan suatu data tertentu di dalam sekumpulan data yang sudah ada. Pencarian merupakan salah satu operasi dasar yang sangat penting dalam pemrograman karena hampir setiap aplikasi membutuhkan kemampuan untuk mencari dan menemukan data. Terdapat dua algoritma pencarian yang paling umum digunakan, yaitu Sequential Search dan Binary Search.

1. Sequential Search

Sequential search atau pencarian berurutan adalah metode pencarian yang paling sederhana. Algoritma ini bekerja dengan cara menelusuri setiap elemen dalam array satu per satu dari indeks pertama hingga indeks terakhir sampai data yang dicari ditemukan. Jika data ditemukan, maka indeks posisinya akan dikembalikan. Jika sudah seluruh elemen diperiksa namun data tidak ditemukan, maka fungsi mengembalikan nilai -1 sebagai tanda bahwa data tidak ada.

Kelebihan sequential search adalah dapat diterapkan pada data yang tidak terurut. Namun kekurangannya adalah kurang efisien untuk data dalam jumlah besar karena kompleksitas waktunya adalah **$O(n)$** , artinya semakin banyak data maka semakin lama proses pencariannya.

2. Binary Search

Binary search atau pencarian biner adalah metode pencarian yang jauh lebih efisien dibandingkan sequential search, namun mensyaratkan bahwa data harus sudah dalam keadaan terurut. Algoritma ini bekerja dengan cara membagi array menjadi dua bagian secara berulang. Pada setiap langkah, nilai elemen tengah dibandingkan dengan data yang dicari. Jika nilainya sama maka data ditemukan, jika data yang dicari lebih besar maka pencarian dilanjutkan ke bagian kanan, dan jika lebih kecil maka pencarian dilanjutkan ke bagian kiri. Proses ini terus berulang hingga data ditemukan atau tidak ada lagi elemen yang bisa diperiksa.

Kompleksitas waktu binary search adalah **$O(\log n)$** , sehingga jauh lebih cepat dibandingkan sequential search untuk data dalam jumlah besar.

B. Guided

1. sequentialSearch.go

```
1  package main
2
3  import "fmt"
4
5  type arrData [5]string //tipe data alias
6
7  func seqSearch(arr arrData, binatangCari string) int {
8      var found int = -1
9      for i := 0; i < len(arr); i++ {
10         if arr[i] == binatangCari {
11             found = i
12             break
13         }
14     }
15     return found
16 }
17
18 func main() {
19     var arrBinatang arrData //array tipe string
20
21     for i := 0; i < len(arrBinatang); i++ {
22         fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
23         fmt.Scan(&arrBinatang[i])
24     }
25     fmt.Println()
26
27     var binatangCari string
28     fmt.Print("Masukkan nama binatang yang mau dicari : ")
29     fmt.Scan(&binatangCari)
30
31     var idxCari int
32     idxCari = seqSearch(arrBinatang, binatangCari)
33
34     if idxCari > -1 {
35         fmt.Printf("Data %s ditemukan pada indeks ke-%d!", binatangCari, idxCari)
36     } else if idxCari == -1 {
37         fmt.Printf("Data %s tidak ditemukan!", binatangCari)
38     }
39
40 }
```

Penjelasan :

Program ini mencari nama binatang dalam sebuah array berisi 5 data string menggunakan algoritma **sequential search**. Program meminta pengguna memasukkan 5 nama binatang, lalu meminta nama binatang yang ingin dicari. Fungsi seqSearch akan menelusuri array satu per satu dari awal hingga menemukan data yang cocok, lalu mengembalikan indeksinya. Jika ditemukan, program mencetak indeks posisinya, dan jika tidak ditemukan program mencetak pesan bahwa data tidak ada.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents
\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Guided\Guided-1\sequentialSearch.go"
Masukkan data binatang indeks ke-0 : Sapi
Masukkan data binatang indeks ke-1 : Kambing
Masukkan data binatang indeks ke-2 : Ayam
Masukkan data binatang indeks ke-3 : Kuda
Masukkan data binatang indeks ke-4 : Gajah

Masukkan nama binatang yang mau dicari : Gajah
Data Gajah ditemukan pada indeks ke-4!
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> 
```

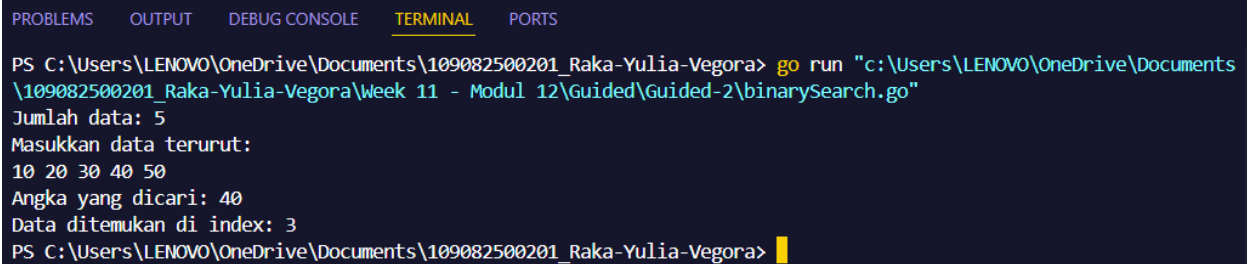
2. binarySearch.go

```
1  package main
2
3  import "fmt"
4
5  // fungsi binary search
6  func binarySearch(data []int, x int) int {
7
8      // batas kiri array
9      kiri := 0
10
11      // batas kanan array
12      kanan := len(data) - 1
13
14      // perulangan selama kiri <= kanan
15      for kiri <= kanan {
16
17          // mencari index tengah
18          tengah := (kiri + kanan) / 2
19
20          // jika data ditemukan
21          if data[tengah] == x {
22              return tengah
23          }
24
25          // jika x lebih besar, cari ke kanan
26          } else if data[tengah] < x {
27              kiri = tengah + 1
28          }
29
30          // jika x lebih kecil, cari ke kiri
31          } else {
32              kanan = tengah - 1
33          }
34      }
35
36      // jika data tidak ditemukan
37      return -1
38  }
39
40  func main() {
41
42      var n int
43      var cari int
44
45      // input jumlah data
46      fmt.Print("Jumlah data: ")
47      fmt.Scan(&n)
48
49      // membuat array sesuai jumlah data
50      data := make([]int, n)
51
52      // input isi array
53      fmt.Println("Masukkan data terurut:")
54      for i := 0; i < n; i++ {
55          fmt.Scan(&data[i])
56      }
57
58      // input angka yang dicari
59      fmt.Print("Angka yang dicari: ")
60      fmt.Scan(&cari)
61
62      // memanggil fungsi binary search
63      hasil := binarySearch(data, cari)
64
65      // output hasil pencarian
66      if hasil != -1 {
67          fmt.Println("Data ditemukan di index:", hasil)
68      } else {
69          fmt.Println("Data tidak ditemukan")
70      }
71  }
```

Penjelasan :

Program ini mencari posisi sebuah angka dalam array integer menggunakan algoritma **binary search**. Program meminta pengguna memasukkan jumlah data, isi array yang sudah terurut, lalu angka yang ingin dicari. Fungsi `binarySearch` bekerja dengan membagi array menjadi dua bagian secara berulang, lalu membandingkan nilai tengah dengan angka yang dicari. Jika nilai tengah lebih kecil maka pencarian dilanjutkan ke kanan, jika lebih besar ke kiri, hingga data ditemukan atau tidak ada. Hasilnya dicetak berupa indeks posisi data atau pesan bahwa data tidak ditemukan.

Output :



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Guided\Guided-2\binarySearch.go"
Jumlah data: 5
Masukkan data terurut:
10 20 30 40 50
Angka yang dicari: 40
Data ditemukan di index: 3
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

3. SearchingStruct.go

```
1 package main
2
3 import "fmt"
4
5 type mahasiswa struct {
6     nama string
7     nim   int
8     IPK   float64
9 }
10
11 type arrMahasiswa [5]mahasiswa
12
13 func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
14     var found int = -1
15     var med int
16     var kr int = 0
17     var kn int = len(arrData) - 1
18     for kr <= kn && found == -1 {
19         med = (kr + kn) / 2
20         if nimDicari < arrData[med].nim {
21             kn = med - 1
22         } else if nimDicari > arrData[med].nim {
23             kr = med + 1
24         } else {
25             found = med
26         }
27     }
28     return found
29 }
30
31 func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
32     var found int = -1
33     for i := 0; i < len(arrData); i++ {
34         if arrData[i].nim == nimDicari {
35             found = i
36             break
37         }
38     }
39     return found
40 }
41
42 func main() {
43     var mahasiswa06 arrMahasiswa
44     var nimDicari int
45
46     fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
47     for i := 0; i < len(mahasiswa06); i++ {
48         fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d ===\n", i)
49         fmt.Print("Masukkan nama : ")
50         fmt.Scan(&mahasiswa06[i].nama)
51         fmt.Print("Masukkan NIM : ")
52         fmt.Scan(&mahasiswa06[i].nim)
53         fmt.Print("Masukkan IPK : ")
54         fmt.Scan(&mahasiswa06[i].IPK)
55         fmt.Println("-----")
56     }
57     fmt.Println()
58
59     fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
60     fmt.Scan(&nimDicari)
61     fmt.Println()
62
63     var idxHasilCariSeqSearch int
64     idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)
65
66     fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
67     if idxHasilCariSeqSearch > -1 {
68         fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
69         fmt.Println("Berikut Detail Datanya : ")
70         fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariSeqSearch].nama)
71         fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariSeqSearch].nim)
72         fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariSeqSearch].IPK)
73     } else if idxHasilCariSeqSearch == -1 {
74         fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
75     }
76     fmt.Println()
77
78     var idxHasilCariBinSearch int
79     idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)
80
81     fmt.Println("== HASIL BINARY SEARCH ==")
82     if idxHasilCariBinSearch > -1 {
83         fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
84         fmt.Println("Berikut Detail Datanya : ")
85         fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariBinSearch].nama)
86         fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariBinSearch].nim)
87         fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariBinSearch].IPK)
88     } else if idxHasilCariBinSearch == -1 {
89         fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
90     }
91 }
```

Penjelasan :

Program ini mencari data mahasiswa berdasarkan NIM menggunakan dua metode pencarian sekaligus, yaitu **sequential search** dan **binary search**. Program meminta pengguna memasukkan data 6 mahasiswa secara terurut berdasarkan NIM, yang masing-masing terdiri dari nama, NIM, dan IPK. Setelah itu pengguna memasukkan NIM yang ingin dicari. Kedua fungsi pencarian akan dijalankan dan hasilnya ditampilkan secara terpisah. Jika data ditemukan, program mencetak detail mahasiswa berupa nama, NIM, dan IPK, sedangkan jika tidak ditemukan program mencetak pesan bahwa data tidak ada.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Guided\Guided-3\SearchingStruct.go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : Dharent
Masukkan NIM : 111
Masukkan IPK : 3.50
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : Raka
Masukkan NIM : 112
Masukkan IPK : 3.70
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : Bintang
Masukkan NIM : 113
Masukkan IPK : 3.75
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : Bulan
Masukkan NIM : 114
Masukkan IPK : 3.60
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : Danis
Masukkan NIM : 115
Masukkan IPK : 3.55
-----

Masukkan NIM mahasiswa yang ingin dicari : 113

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 113 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Bintang
NIM : 113
IPK : 3.75

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 113 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Bintang
NIM : 113
```

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data harus divalidasi. Tugas Anda membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program pilkart yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara

Jawaban :

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      votes := make(map[int]int)
7      count := 0
8      valid := 0
9
10     for {
11         var n int
12         fmt.Scan(&n)
13         if n == 0 {
14             break
15         }
16         count++
17         if n >= 1 && n <= 20 {
18             valid++
19             votes[n]++
20         }
21     }
22
23     fmt.Printf("Suara masuk: %d\n", count)
24     fmt.Printf("Suara sah: %d\n", valid)
25
26     for i := 1; i <= 20; i++ {
27         if votes[i] > 0 {
28             fmt.Printf("%d: %d\n", i, votes[i])
29         }
30     }
31 }
```

Penjelasan :

Program ini adalah sistem penghitung suara pemilihan ketua RT. Program membaca angka satu per satu secara terus-menerus hingga menemukan angka 0 sebagai tanda berhenti. Setiap angka yang dibaca akan dihitung sebagai suara masuk, namun hanya angka antara 1 sampai 20 yang dianggap sebagai suara sah dan dicatat ke dalam map votes. Setelah input selesai, program mencetak total suara masuk, total suara sah, serta perolehan suara tiap kandidat yang mendapatkan minimal satu suara.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Unguided\Unguided-no1\pemilihanKetuaRT.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> |
```

2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT baru.

Jawaban :

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      votes := make(map[int]int)
7      totalInput := 0
8      validVotes := 0
9
10     for {
11         var n int
12         fmt.Scan(&n)
13         if n == 0 {
14             break
15         }
16         totalInput++
17         if n >= 1 && n <= 20 {
18             votes[n]++
19             validVotes++
20         }
21     }
22
23     fmt.Printf("Suara masuk: %d\n", totalInput)
24     fmt.Printf("Suara sah: %d\n", validVotes)
25
26     // Cari suara terbanyak pertama dan kedua
27     firstMax := 0
28     secondMax := 0
29
30     for _, count := range votes {
31         if count > firstMax {
32             secondMax = firstMax
33             firstMax = count
34         } else if count > secondMax {
35             secondMax = count
36         }
37     }
38
39     // Cari kandidat ketua (suara terbanyak, nomor terkecil jika sama)
40     ketua := -1
41     for candidate := 1; candidate <= 20; candidate++ {
42         if votes[candidate] == firstMax {
43             if ketua == -1 {
44                 ketua = candidate
45             }
46         }
47     }
48
49     // Cari kandidat wakil (suara terbanyak kedua, nomor terkecil jika sama)
50     // Tapi bukan kandidat ketua
51     wakil := -1
52     // Hitung berapa kandidat dengan suara firstMax
53     countFirst := 0
54     for _, count := range votes {
55         if count == firstMax {
56             countFirst++
57         }
58     }
59
60     if countFirst >= 2 {
61         // Ada tie di posisi pertama, wakil = kandidat terkecil berikutnya dengan firstMax
62         for candidate := 1; candidate <= 20; candidate++ {
63             if votes[candidate] == firstMax && candidate != ketua {
64                 wakil = candidate
65                 break
66             }
67         }
68     } else {
69         // Wakil dari suara terbanyak kedua
70         for candidate := 1; candidate <= 20; candidate++ {
71             if votes[candidate] == secondMax {
72                 wakil = candidate
73                 break
74             }
75         }
76     }
77
78     fmt.Printf("Ketua RT: %d\n", ketua)
79     fmt.Printf("Wakil ketua: %d\n", wakil)
80 }

```

Penjelasan :

Program ini menentukan ketua dan wakil ketua RT berdasarkan hasil voting. Program membaca angka satu per satu hingga menemukan 0, lalu menghitung suara masuk dan suara sah (angka 1–20). Setelah itu, program mencari kandidat dengan suara terbanyak sebagai ketua, dan suara terbanyak kedua sebagai wakil. Jika ada kandidat dengan suara yang sama, dipilih yang nomor pesertanya paling kecil. Hasil akhir yang dicetak adalah total suara masuk, suara sah, nomor ketua, dan nomor wakil ketua.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Unguided\Unguided-no2\pemenangpemilihanRT.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang dicari. Baris kedua berisi n buah data integer positif yang terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam Kumpulan.

Jawaban :

```

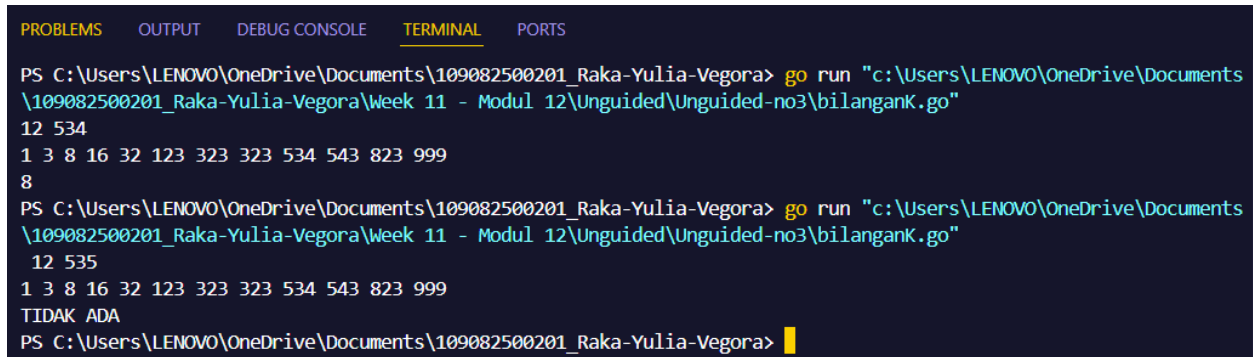
1  package main
2
3  import "fmt"
4
5  const NMAX = 1000000
6
7  var data [NMAX]int
8
9  func main() {
10     var n, k int
11     fmt.Scan(&n, &k)
12
13     isiArray(n)
14
15     result := posisi(n, k)
16
17     if result == -1 {
18         fmt.Println("TIDAK ADA")
19     } else {
20         fmt.Println(result)
21     }
22 }
23
24 func isiArray(n int) {
25     for i := 0; i < n; i++ {
26         fmt.Scan(&data[i])
27     }
28 }
29
30 func posisi(n, k int) int {
31     // Binary search karena data sudah terurut membesar
32     low := 0
33     high := n - 1
34
35     for low <= high {
36         mid := (low + high) / 2
37         if data[mid] == k {
38             return mid
39         } else if data[mid] < k {
40             low = mid + 1
41         } else {
42             high = mid - 1
43         }
44     }
45
46     return -1
47 }

```

Penjelasan :

Program ini mencari posisi sebuah bilangan k dalam array yang sudah terurut membesar. Program membaca input n dan k, lalu mengisi array melalui fungsi isiArray. Pencarian dilakukan oleh fungsi posisi menggunakan algoritma **binary search**, yaitu dengan membandingkan nilai tengah array secara berulang hingga nilai k ditemukan. Jika ditemukan, program mencetak indeks posisinya (mulai dari 0), dan jika tidak ditemukan program mencetak "TIDAK ADA".

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Unguided\Unguided-no3\bilangank.go"
12 534
1 3 8 16 32 123 323 534 543 823 999
8
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 11 - Modul 12\Unguided\Unguided-no3\bilangank.go"
12 535
1 3 8 16 32 123 323 534 543 823 999
TIDAK ADA
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

4. Kesimpulan

Dari praktikum Modul 12 tentang Searching ini, dapat disimpulkan bahwa algoritma pencarian merupakan bagian penting dalam pemrograman yang digunakan untuk menemukan data tertentu dalam sekumpulan data. Terdapat dua algoritma pencarian utama yang dipelajari, yaitu sequential search dan binary search, yang masing-masing memiliki karakteristik dan kegunaan yang berbeda.

Sequential search bekerja dengan menelusuri data satu per satu dari awal hingga akhir sehingga dapat diterapkan pada data yang tidak terurut, namun kurang efisien untuk data berukuran besar karena memiliki kompleksitas waktu $O(n)$. Sementara itu, binary search bekerja dengan membagi data menjadi dua bagian secara berulang sehingga jauh lebih efisien dengan kompleksitas waktu $O(\log n)$, namun mensyaratkan data harus sudah dalam keadaan terurut.

Pada program unguided, kedua konsep tersebut diterapkan dalam studi kasus nyata seperti sistem penghitungan suara pemilihan ketua RT dan pencarian bilangan dalam array besar. Pada program guided, algoritma pencarian diterapkan pada berbagai tipe data mulai dari string, integer, hingga struct mahasiswa. Dari seluruh praktikum ini dapat disimpulkan bahwa pemilihan algoritma pencarian yang tepat sangat bergantung pada kondisi data, di mana binary search lebih unggul untuk data terurut berukuran besar, sedangkan sequential search lebih fleksibel untuk data yang tidak terurut.